Chapter 6

Data Encryption Standard (DES)

Chapter 6

Objectives

- To review a short history of DES
- To define the basic structure of DES
- To describe the details of building elements of DES
- To describe the round keys generation process
- To analyze DES

Block vs Stream Ciphers

- block ciphers process messages in into blocks, each of which is then en/decrypted.
- like a substitution on very big characters
  - 64-bits or more
- stream ciphers process messages a bit or byte at a time when en/decrypting
- many current ciphers are block ciphers
- hence are focus of course

6-1 INTRODUCTION

- *Data Encryption Standard (DES)*
- *is a symmetric-key block cipher*
- *published by the*
  - *National Institute of Standards and Technology (NIST).*

Topics discussed in this section:

6.1.1 History

6.1.2 Overview

6.1.1 History

- *In 1973, NIST published a request for proposals for a national symmetric-key cryptosystem.*

- *A proposal from IBM, a modification of a project called Lucifer, was accepted as DES.*

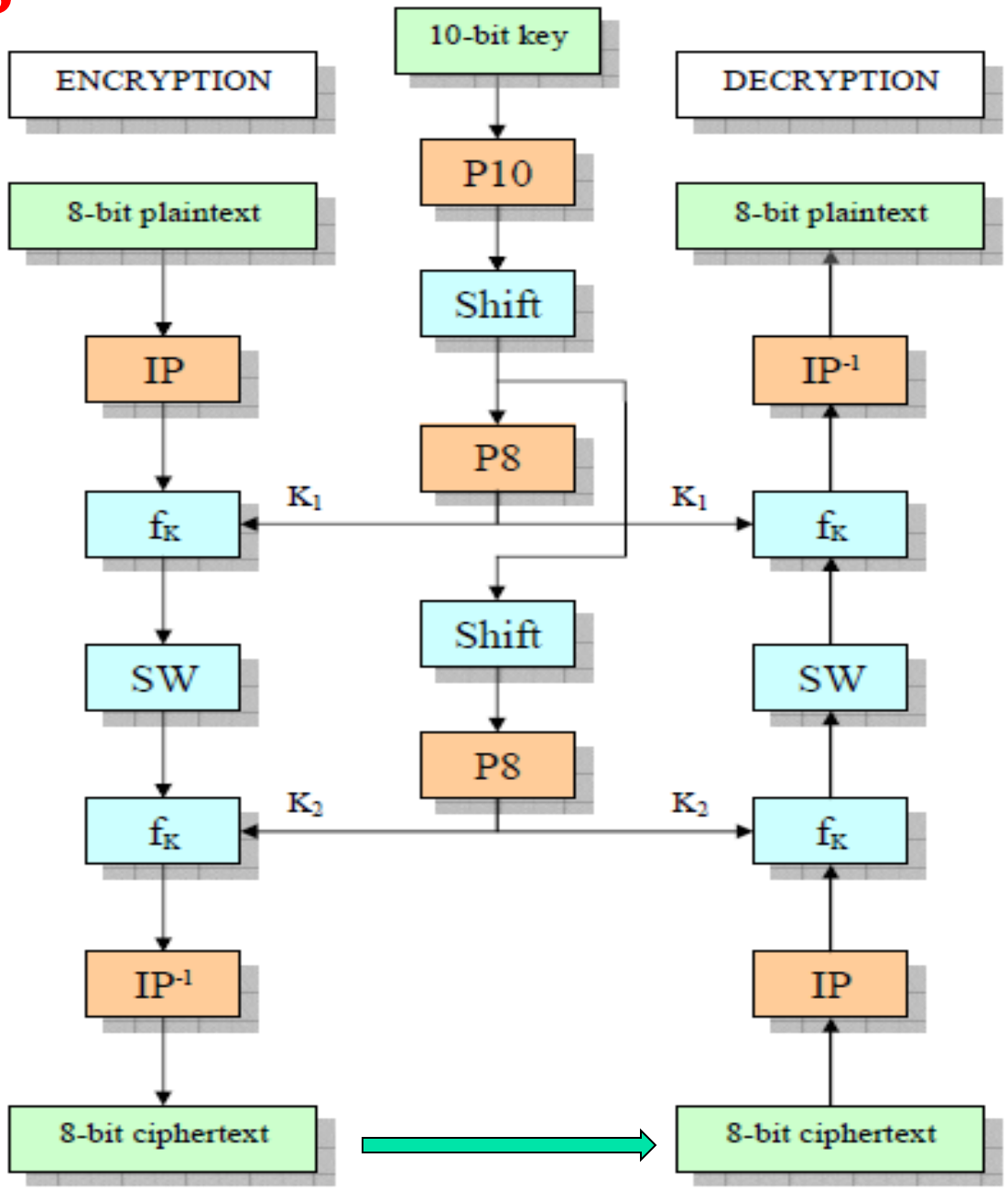
- *DES was published in the Federal Register in March 1975 as a draft of the Federal Information Processing Standard (FIPS).*

Simplified DES

- an educational algorithm rather than a secure algorithm.

- **Involves five functions:**
  - An initial permutation (**IP**).
  - A complex function labeled f_k, which involves both permutation and substitution operations and depends on a key input.
  - A simple permutation function that switches (**SW**) the two halves of the data.
  - The function f_k applied again.
  - A permutation function that is the inverse of the initial permutation (IP^{-1}).

S-DES


ENCRYPTION
10-bit key
DECRYPTION
P10
8-bit plaintext
8-bit plaintext
Shift
IP
IP-1
P8
K1
K1
fK
fK
Shift
SW
SW
P8
K2
K2
fK
fK
IP-1
IP
8-bit ciphertext
8-bit ciphertext

Simplified DES

We can express encryption algorithm as superposition:

$$IP^{-1} \circ f_{K_2} \circ SW \circ f_{K_1} \circ IP$$

or

Ciphertext= $IP^{-1}\left(f_{K_2}(SW(f_{K_1}(IP(plaintext)))))\right)$

And the

Plaintext= $IP^{-1}\left(f_{K_1}(SW(f_{K_2}(IP(ciphertext)))))\right)$

S-DES Key Generation:

- S-DES depends on the use of a 10-bit key shared between sender and receiver.
- From this key, two 8-bit subkeys are produced for use in particular stages of the encryption and decryption algorithm.
  - K_1 = P8(Shift(P10(key))
  - K_2 = P8(Shift(Shift(P10(key)))

S-DES Key Generation

First, permute the key in the following fashion

P10									
3	5	2	7	4	10	1	9	8	6

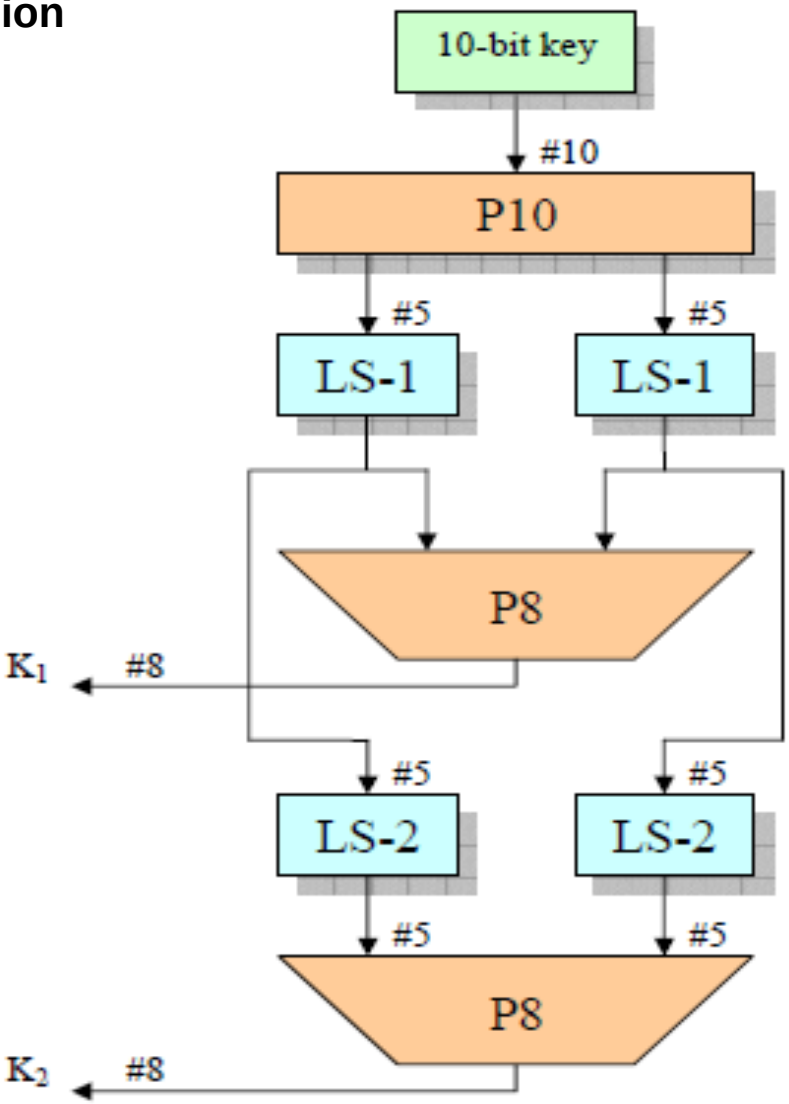
Key - 1010000010

Step 1: For example, the key (1010000010) is permuted to (1000001100).

Step 2: Divide (1000001100) into a left part 5-bit value (10000) and a right part 5-bit value (01100).

Step 3: Perform a circular left shift (LS-1), or rotation, separately. The left value (10000) becomes (00001). The right value (01100) becomes (11000). Concatenate the left part (00001) and the right part (11000) into a 10-bit value (0000111000).

Step 4: Pick out and permutes 8, (don't use 1 and 2), of the 10 bits according to the following rules:



P8							
6	3	7	4	8	5	10	9

Continued.

The result is subkey k1. The value (00 00111000) becomes:

k1 = (10100100).

Step 5: Go back to the pair of 5-bit strings produced by the two LS-1 functions, the results of step 3, and perform a circular left shift of 2 bit positions on each string. The left value (00001) becomes (00100) and the right value (11000) becomes (00011).

Concatenate the left part (00100) and the right part (00011) into a 10-bit value (0010000011).

Step 6: Finally, P8 is applied again to produce k2, the value (00 10000011) becomes:

k2 = (01000011).

S-DES ENCRYPTION/ DECRYPTION DETAILS

Example:

key = [1010000010]

plaintext

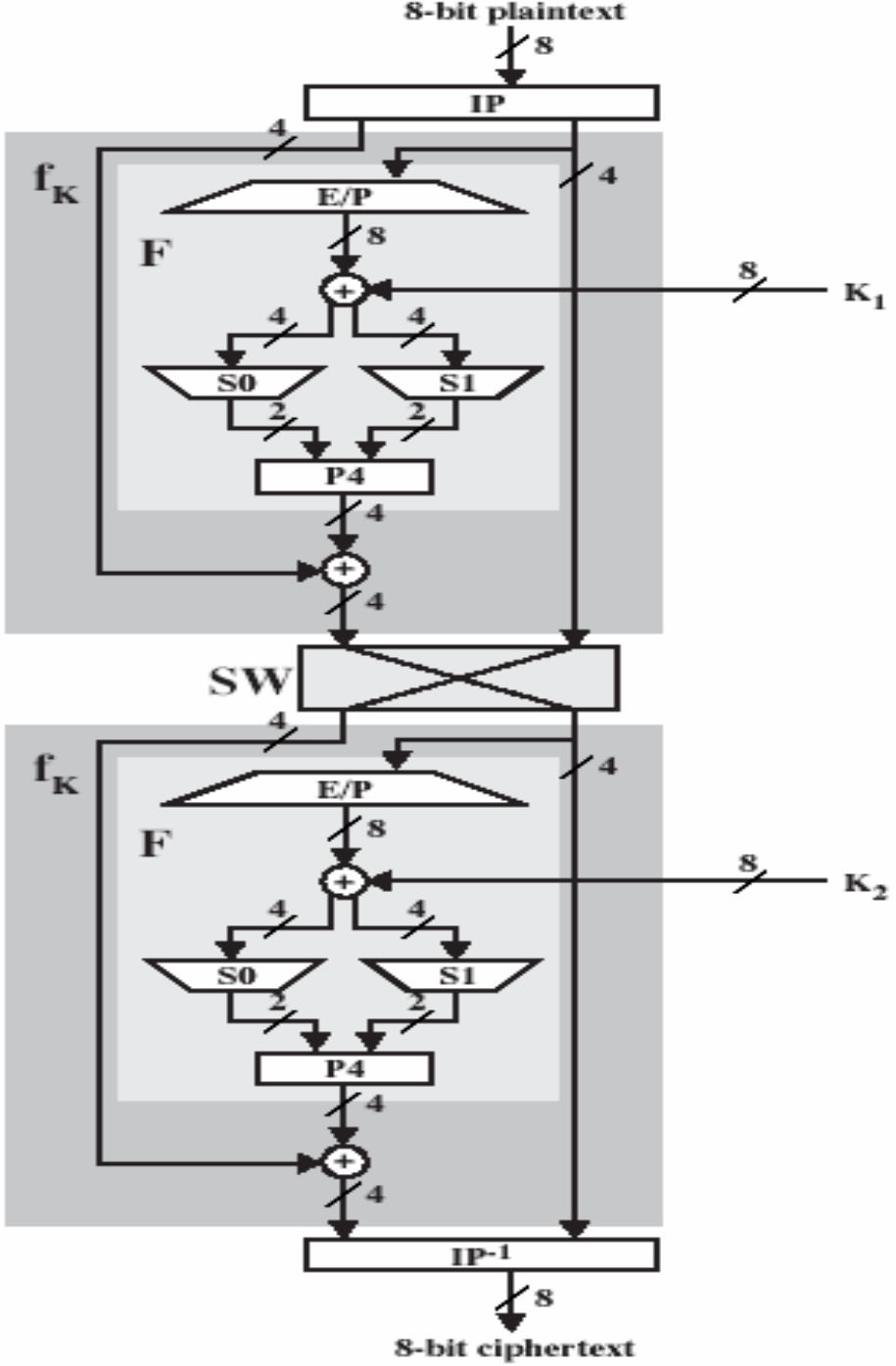
= [01000001]

= A (char)

ciphertext

= [00010101]

= 15 (hex)



Initial and Final Permutations

- The input to the algorithm is an 8-bit block of plaintext (10111101), which we first permute using the IP function.

- **Step first**: Permute the plaintext (**10111101**) with IP and the value becomes (01111110).

IP							
2	6	3	1	4	8	5	7

Step last: Permute the value (11101100) with IP^{-1} and the ciphertext becomes (01110101).

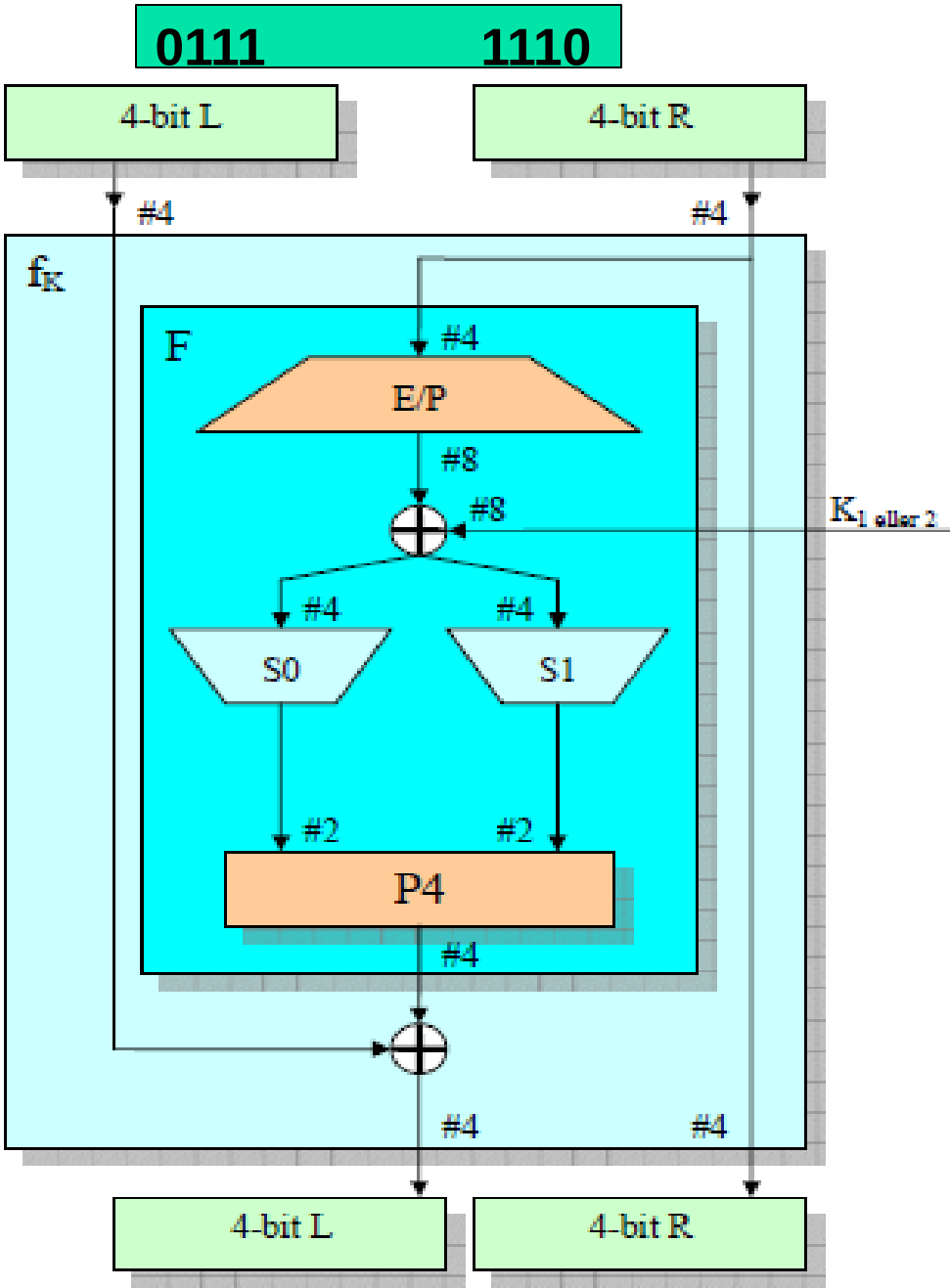
IP^{-1}							
4	1	3	5	7	2	8	6

It is easy to show by example that the second permutation is indeed the reverse of the first; that is, $IP^{-1}(IP(X)) = X$.

Details of one Round

Step one:

- **Divide the value into two parts.**
- **The left part becomes (0111) and the right part becomes (1110).**



The Function F and f_k

The most complex component of S-DES is the function f_k, which consists of a combination of permutation and substitution functions

$$f_k(L,R) = (L \oplus F(R,SK),R)$$

E/P:Expantion/Permutation

Step 2:

Permute using E/P, the Right half(1110) and we have (01111101).

E/P							
4	1	2	3	2	3	4	1

The XOR

Step 3:

Create a matrix based on the result of step 2.
Row 1 is (0111). Row 2 is (1101).

n_4	n_1	n_2	n_3
n_2	n_3	n_4	n_1

0	1	1	1
1	1	0	1

Step 4:

Create a matrix based on the 8-bit subkey

K1 = (k11, k12, k13, k14, k15, k16, k17,k18).

We use the key (1 0 1 0 0 1 0 0).

k_{11}	k_{12}	k_{13}	k_{14}
k_{15}	k_{16}	k_{17}	k_{18}

1	0	1	0
0	1	0	0

Step 5:Perform an exclusive-OR function on the matrix in step 3 and step 4.

$n_4 \oplus k_1$	$n_1 \oplus k_2$	$n_2 \oplus k_3$	$n_3 \oplus k_4$
$n_2 \oplus k_5$	$n_3 \oplus k_6$	$n_4 \oplus k_7$	$n_3 \oplus k_8$

1	1	0	1
1	0	0	1

The XOR Continued.

Step 6:

Rename the result of matrix in step 5. Create groups according to these roles: ($P_{0,0}$ $P_{0,3}$), ($P_{0,1}$ $P_{0,2}$), ($P_{1,0}$ $P_{1,3}$) and ($P_{1,1}$ $P_{1,2}$). Convert the binary value to decimal.

$P_{0,0}$	$P_{0,1}$	$P_{0,2}$	$P_{0,3}$
$P_{1,0}$	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$

1	1	0	1
1	0	0	1

$(P_{0,0}\ P_{0,3}) = (11) = 3$

$(P_{0,1}\ P_{0,2}) = (10) = 2$

$(P_{1,0}\ P_{1,3}) = (11) = 3$

$(P_{1,1}\ P_{1,2}) = (00) = 0$

S-boxes

Step 8: The S-boxes operate as follows. The first and fourth input bits are treated as a 2-bit number that specify a row of the S-box, and the second and third input bits specify a column of the S-box. The entry in that row and column, in base 2, is the 2-bit row 0,column 2 of S0, which is 3, or (11) in binary. Add the two 2-bit results into a one 4-bit value.

$S0_y = (P_{0,0}\ P_{0,3}) = (11) = 3$
$S0_x = (P_{0,1}\ P_{0,2}) = (10) = 2$
$S0(y,x) = 3\ (dec) = 11\ (bin)$

$S1_y = (P_{1,0}\ P_{1,3}) = (11) = 3$
$S1_x = (P_{1,1}\ P_{1,2}) = (00) = 0$
$S1(y,x) = 2\ (dec) = 10\ (bin)$

Concatenate S0 (3=11) and S1 (2=10) into a 4-bit value (1110).

S0 =

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	**3**	2

S1 =

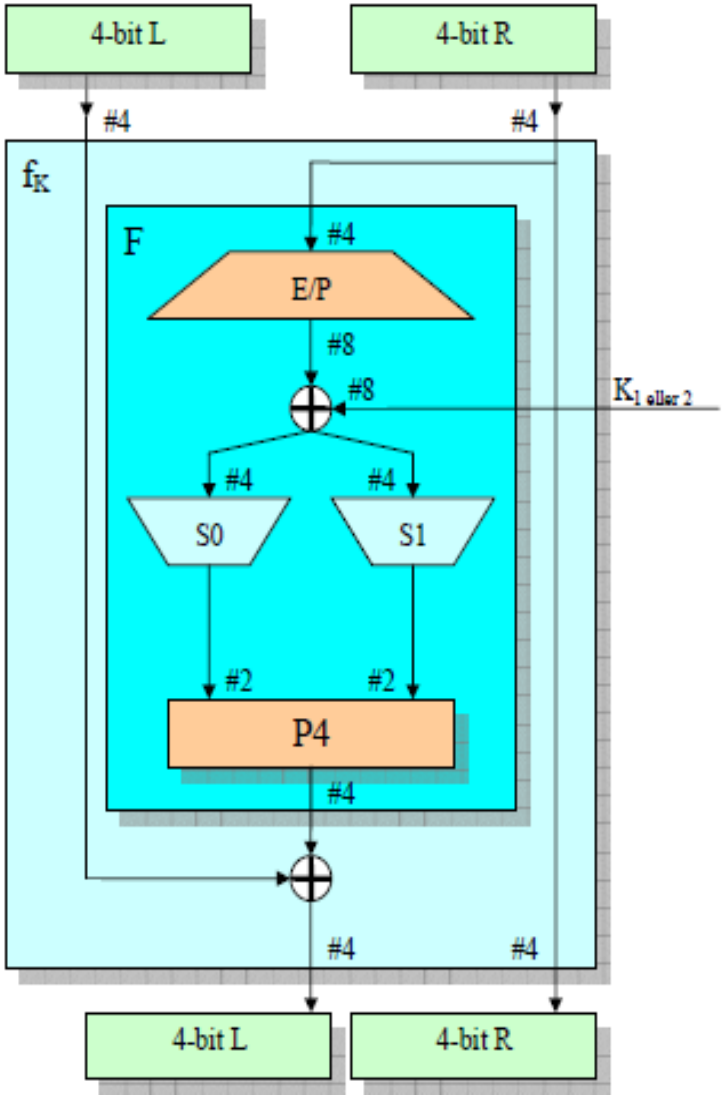
	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	**2**	1	0	3

Continued

Step 8:

Permute the value (1110) which becomes (1011) using P4.

P4			
2	4	3	1



Step 9:

Take the left value from step 1 (0111) and exclusive-OR the value from step 9 (1011).
(0111) ⊕ (1011) = (1100).

Step 10:

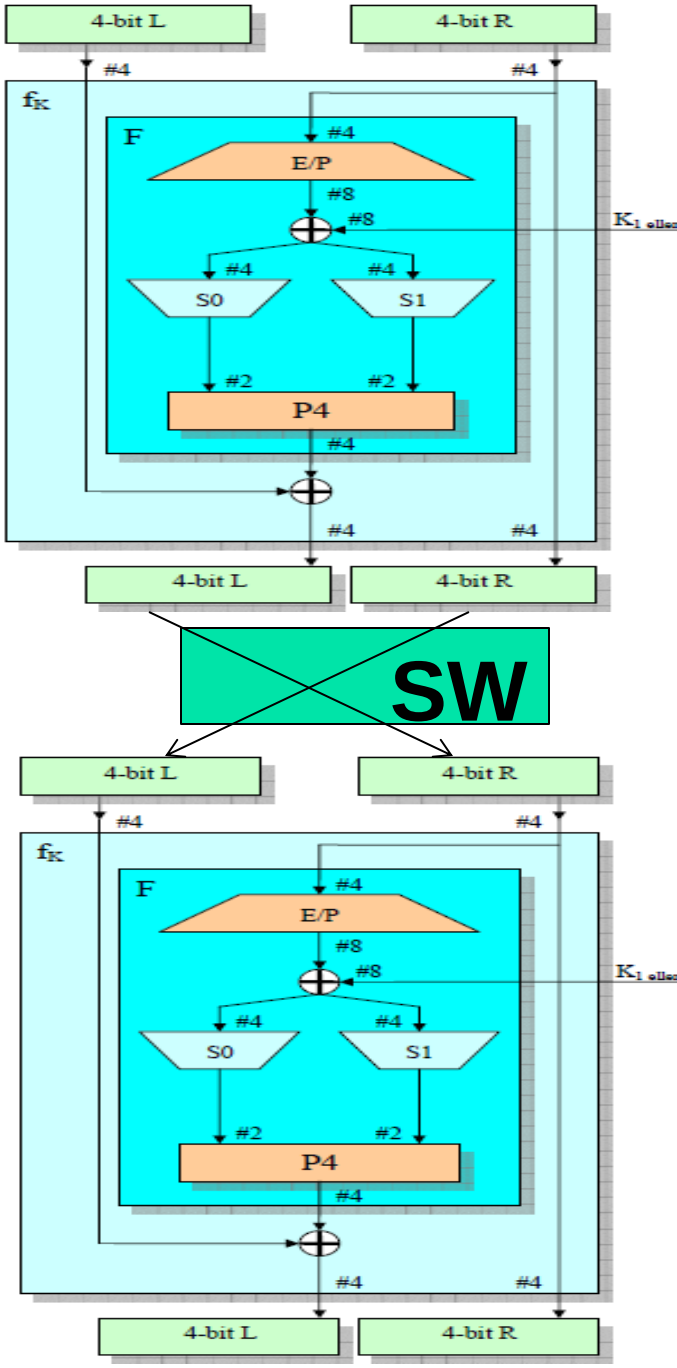
Take the value from step 10 (1100) and the right value from step 1 (1110).

The Switch Function

- Take the value from step 9 (1100) and the right 4-bit value from IP (1110).

- Swap places with each other.

- Use (1110) as input to the left side in the f_k in the second Round.
- Use (1100) as input to the right side in the f_k in the second Round.



Continued

- **Follow the above steps and continue with the second round of operations.**
- **At end IP^{-1} is applied again to find the ciphertext**

key = [1010000010]
plaintext = [01000001] = A (char)
Ciphertext = [00010101] = 15 (hex)

EXAMPLE:

IP							
2	6	3	1	4	8	5	7

IP^{-1}							
4	1	3	5	7	2	8	6

P10									
3	5	2	7	4	10	1	9	8	6

P8							
6	3	7	4	8	5	10	9

P4			
2	4	3	1

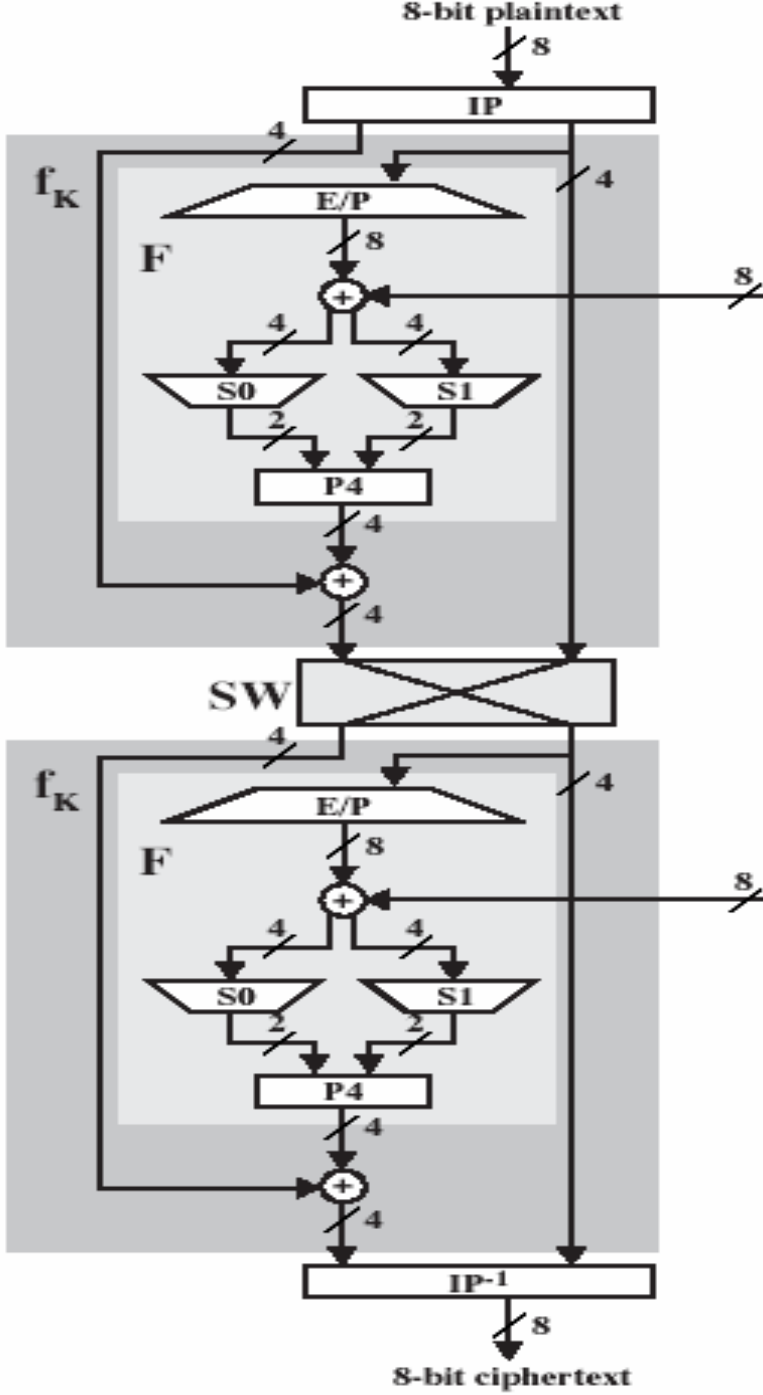
E/P							
4	1	2	3	2	3	4	1

S0 =

	0	1	2	3
0	1	0	3	2
1	3	2	1	0
2	0	2	1	3
3	3	1	**3**	2

S1 =

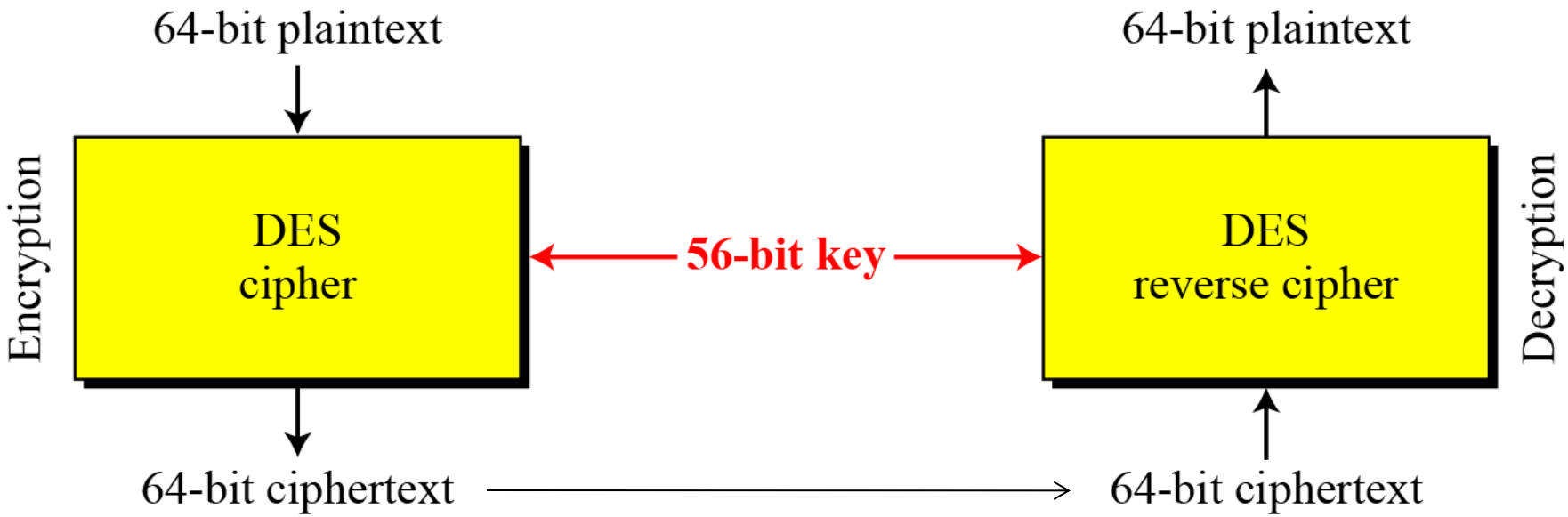
	0	1	2	3
0	0	1	2	3
1	2	0	1	3
2	3	0	1	0
3	**2**	1	0	3



6.1.2 Overview

DES is a block cipher, as shown in Figure 6.1.

Figure 6.1 *Encryption and decryption with DES*



6-2 DES STRUCTURE

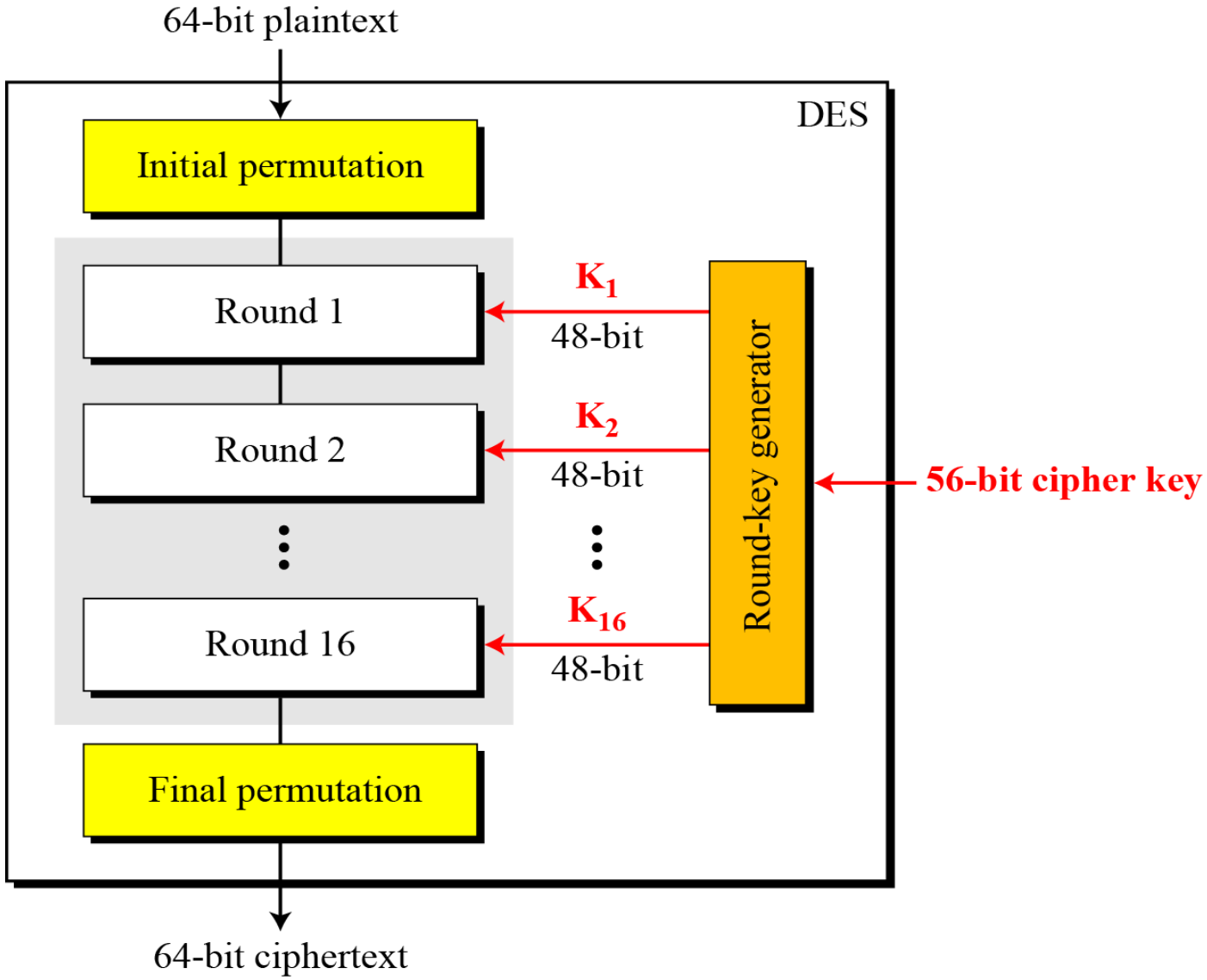
The encryption process is made of two permutations (P-boxes), which we call initial and final permutations, and sixteen Feistel rounds.

Topics discussed in this section:

6.2.1 Initial and Final Permutations
6.2.2 Rounds
6.2.3 Cipher and Reverse Cipher
6.2.4 Examples

6-2 Continue

Figure 6.2 *General structure of DES*

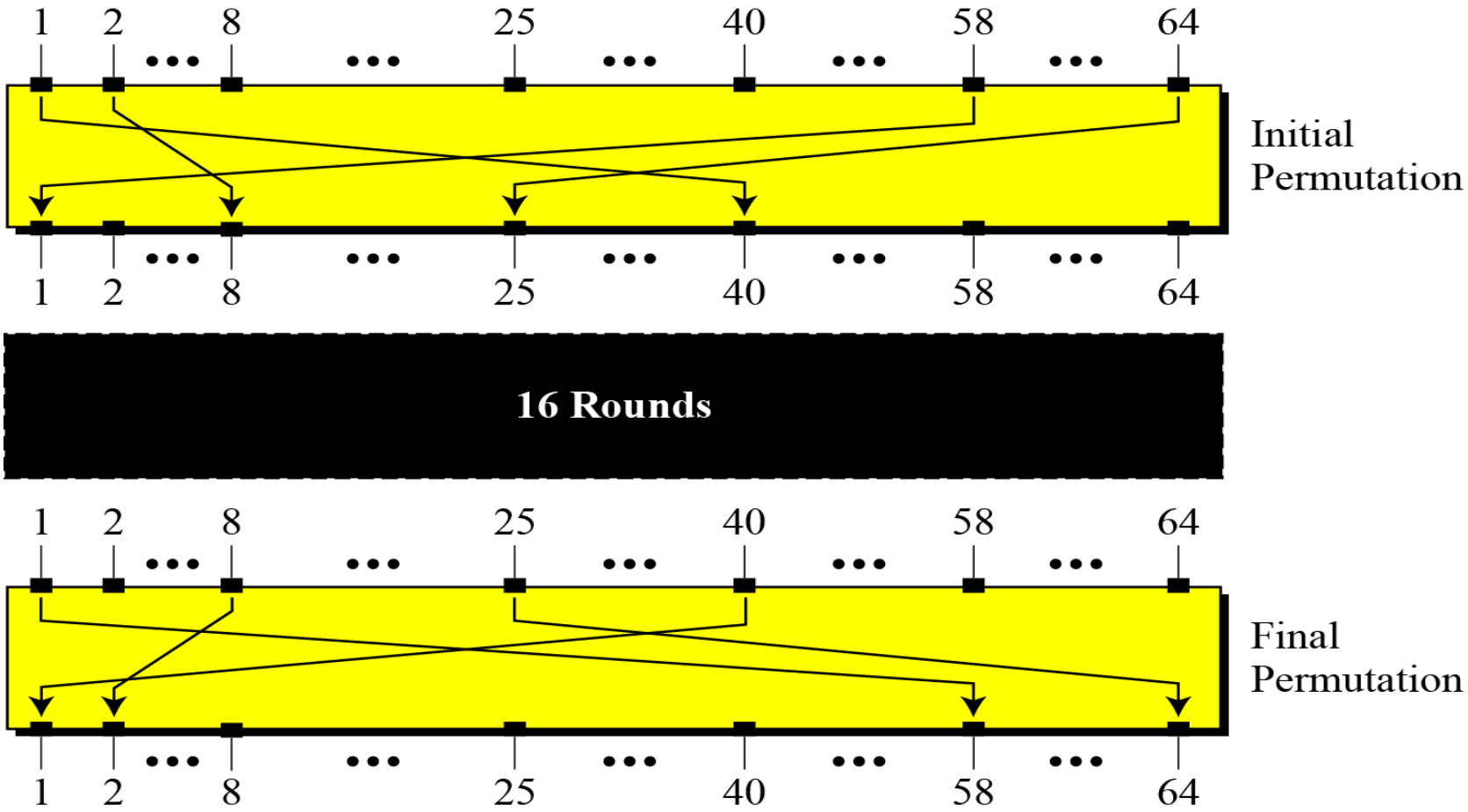


DES Key Schedule

- forms 16 subkeys, used in each round
- consists of:
  - A shared secret key of 64 bits
    - 8,16,24,32,40,48,56,64th bits are parity bits
  - initial permutation of the key (PC1) which selects 56-bits in two 28-bit halves
  - 16 stages consisting of:
    - selecting 24-bits from each half
    - permuting them by PC2 for use in function f,
    - rotating **each half** separately either 1 or 2 places depending on the **key rotation schedule** K

6.2.1 Initial and Final Permutations

Figure 6.3 *Initial and final permutation steps in DES*

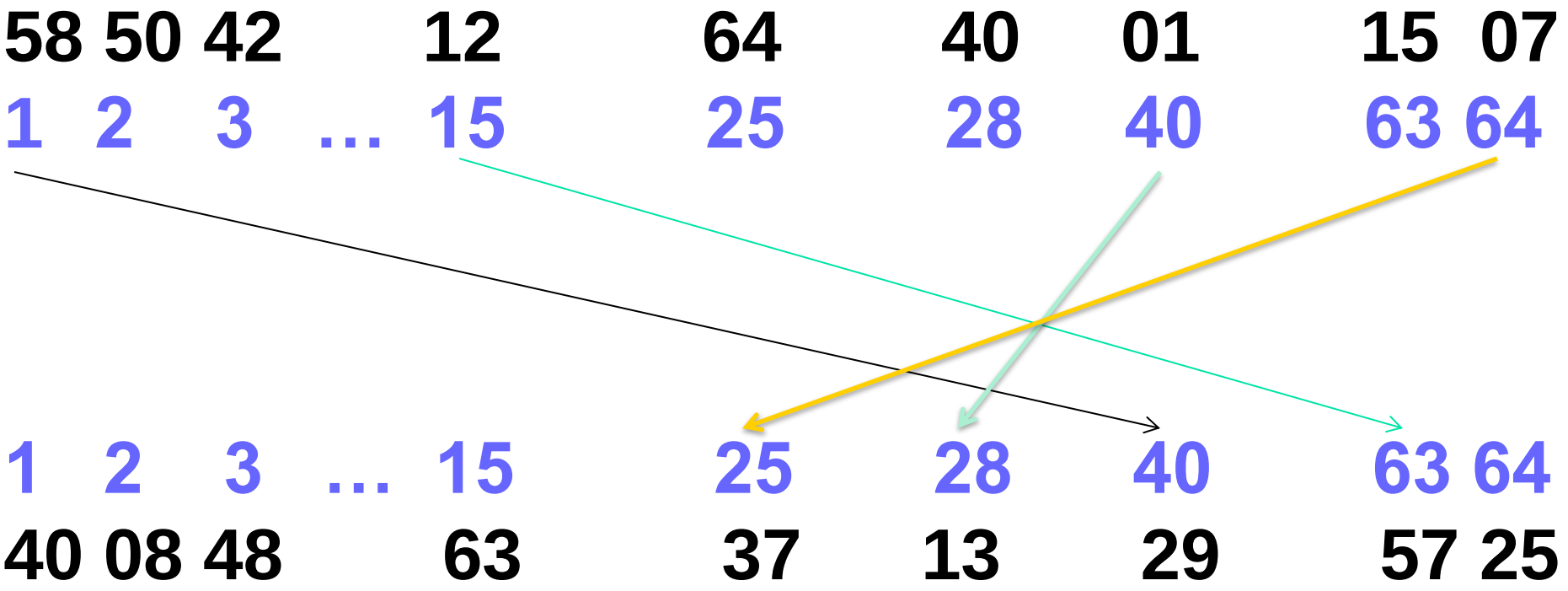


6.2.1 Continue

Table 6.1 *Initial and final permutation tables*

Initial Permutation	*Final Permutation*
58 50 42 34 26 18 10 02	40 08 48 16 56 24 64 32
60 52 44 36 28 20 12 04	39 07 47 15 55 23 63 31
62 54 46 38 30 22 14 06	38 06 46 14 54 22 62 30
64 56 48 40 32 24 16 08	37 05 45 13 53 21 61 29
57 49 41 33 25 17 09 01	36 04 44 12 52 20 60 28
59 51 43 35 27 19 11 03	35 03 43 11 51 19 59 27
61 53 45 37 29 21 13 05	34 02 42 10 50 18 58 26
63 55 47 39 31 23 15 07	33 01 41 09 49 17 57 25

Initial and final permutation tables



Example 6.1

Find the output of the initial permutation box when the input is given in hexadecimal as:

0x0000 0080 0000 0002

Solution:

Only bit 25 and bit 64 are 1s; the other bits are 0s. In the final permutation, bit 25 becomes bit 64 and bit 63 becomes bit 15. The result is

0x0002 0000 0000 0001

6.2.1 Continued

Example 6.2

Prove that the initial and final permutations are the inverse of each other by finding the output of the final permutation if the input is

0x0002 0000 0000 0001

Solution

The input has only two 1s; the output must also have only two 1s. Using Table 6.1, we can find the output related to these two bits. Bit 15 in the input becomes bit 63 in the output. Bit 64 in the input becomes bit 25 in the output. So the output has only two 1s, bit 25 and bit 63. The result in hexadecimal is

0x0000 0080 0000 0002

Note

The initial and final permutations are straight P-boxes that are inverses of each other.
They have no cryptography significance in DES.

6.2.2 Rounds

DES uses 16 rounds. Each round of DES is a Feistel cipher.

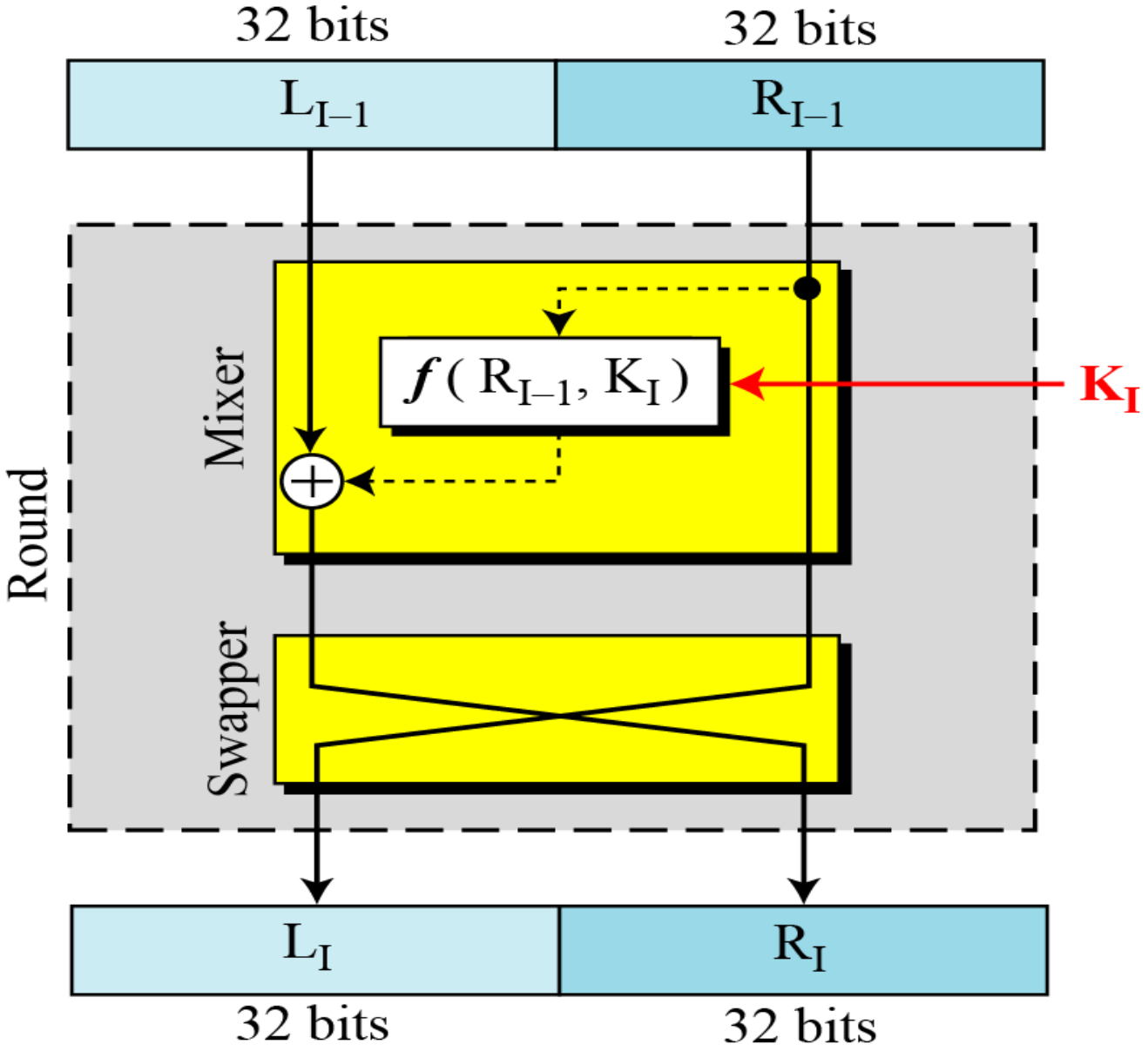


Figure 6.4

A round in DES (encryption site)

6.2.2 Continued

DES Function

- *The heart of DES is the DES function.*
- *The DES function applies a 48-bit key to the rightmost 32 bits to produce a 32-bit output.*

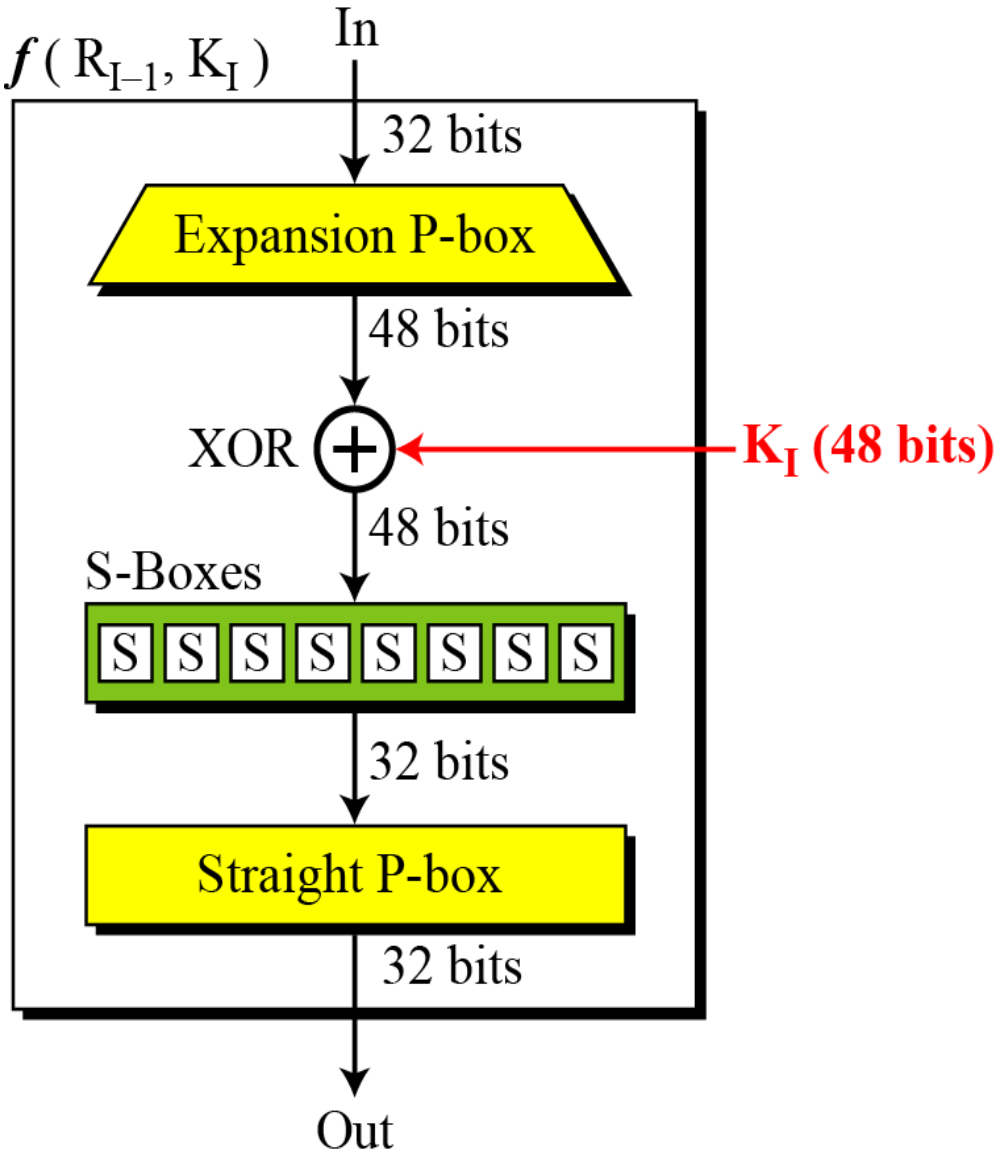


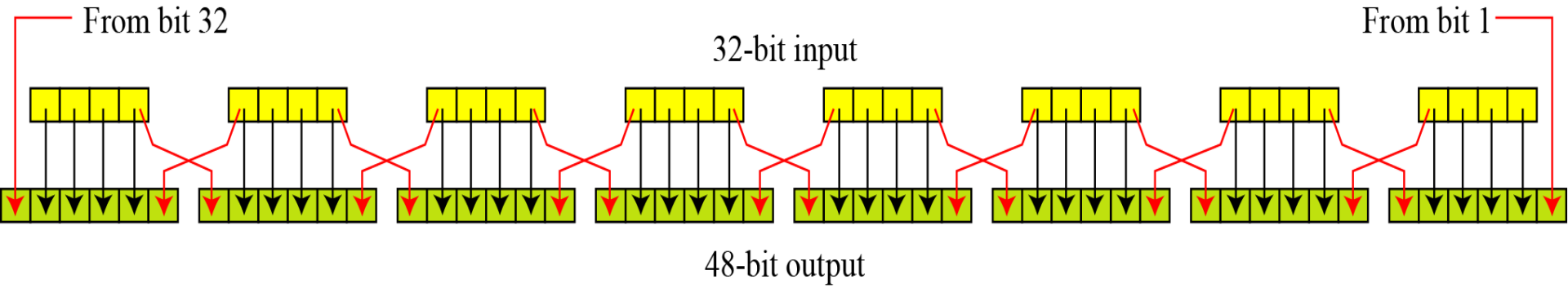
Figure 6.5 *DES function*

6.2.2 Continue

Expansion P-box

Since R_{I-1} is a 32-bit input and K_I is a 48-bit key, we first need to expand R_{I-1} to 48 bits.

Figure 6.6 *Expansion permutation*



6.2.2 Continue

Although the relationship between the input and output can be defined mathematically, DES uses Table 6.2 to define this P-box.

Table 6.6 *Expansion P-box table*

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

6.2.2 Continue

Whitener (XOR)

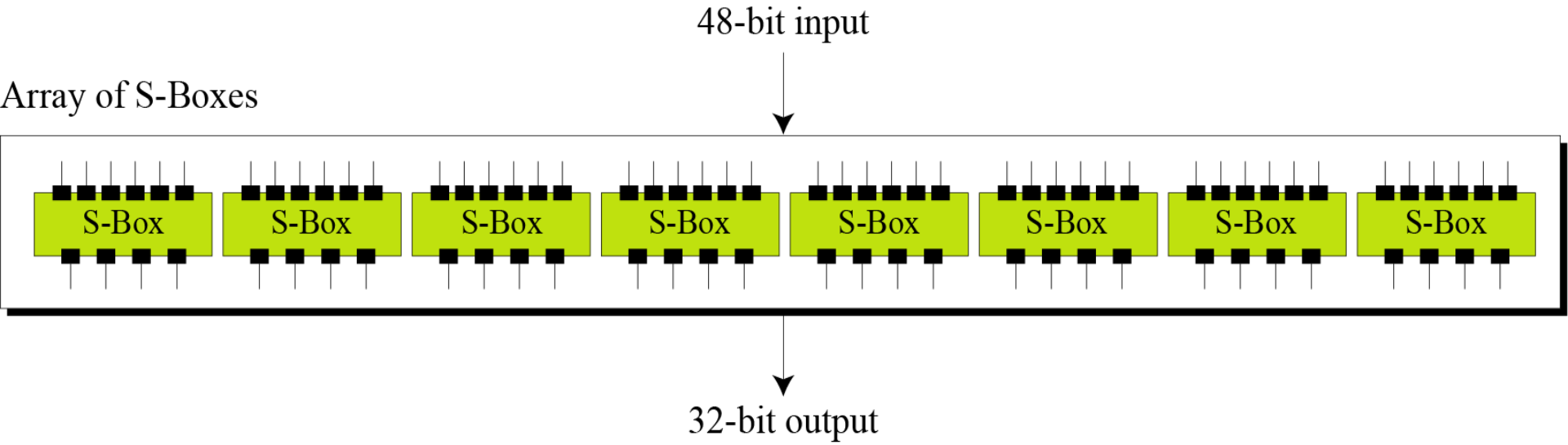
After the expansion permutation, DES uses the XOR operation on the expanded right section and the round key.

Note that both the right section and the key are 48-bits in length. Also note that the round key is used only in this operation.

S-Boxes

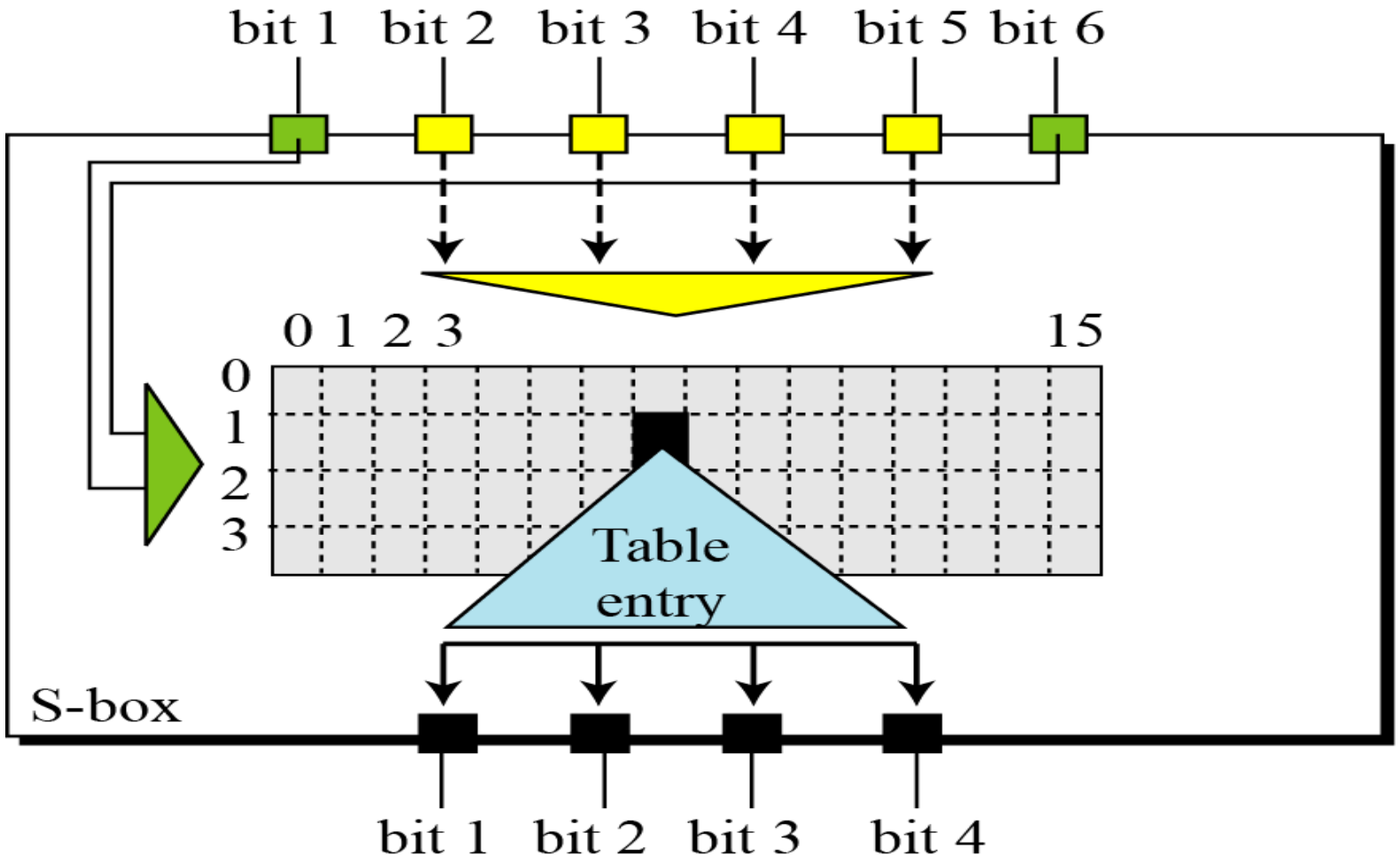
The S-boxes do the real mixing (confusion). DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output. See Figure 6.7.

Figure 6.7 *S-boxes*



6.2.2 Continue

Figure 6.8 *S-box rule*



6.2.2 Continue

Table 6.3 shows the permutation for S-box 1. For the rest of the boxes see the textbook.

Table 6.3 *S-box 1*

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Q:The input to S-box 1 is 100011.
What is the output?

Example 6.3

The input to S-box 1 is 100011. What is the output?

Solution

If we write the first and the sixth bits together, we get 11 in binary, which is 3 in decimal.
The remaining bits are 0001 in binary, which is 1 in decimal.

We look for the value in row 3, column 1, in Table 6.3 (S-box 1). The result is 12 in decimal, which in binary is 1100. So the input 100011 yields the output 1100.

6.2.2 Continued

Example 6.4

The input to S-box 8 is 000000. What is the output?

Solution

If we write the first and the sixth bits together, we get 00 in binary, which is 0 in decimal. The remaining bits are 0000 in binary, which is 0 in decimal. We look for the value in row 0, column 0, in Table 6.10 (S-box 8). The result is 13 in decimal, which is 1101 in binary. So the input 000000 yields the output 1101.

Straight Permutation

Table 6.11 *Straight permutation table*

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

6.2.3 Cipher and Reverse Cipher

Using mixers and swappers, we can create the cipher and reverse cipher, each having 16 rounds.

First Approach

To achieve this goal, one approach is to make the last round (round 16) different from the others; it has only a mixer and no swapper.

Note

In the first approach, there is no swapper in the last round.

Continued

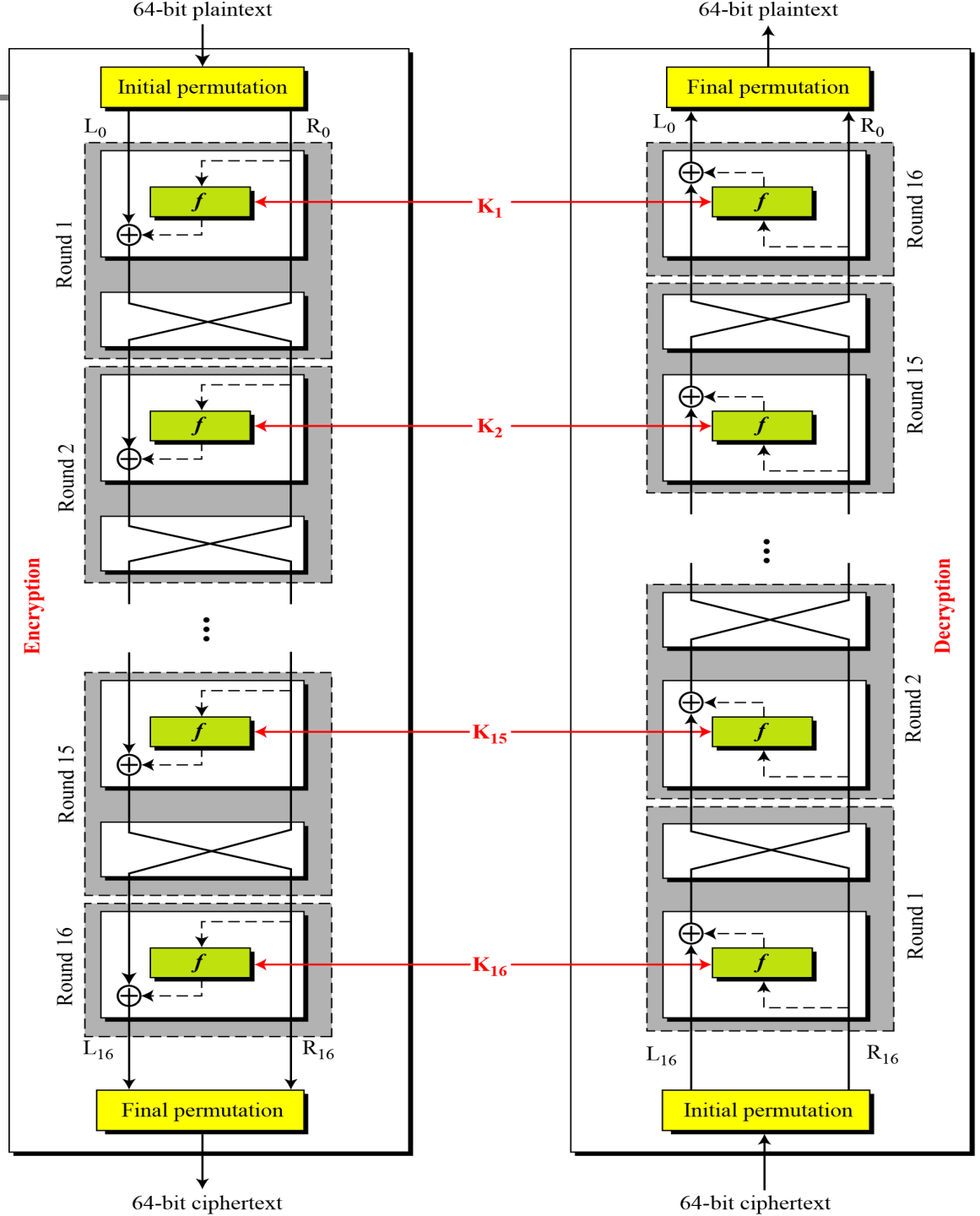


Figure 6.9 ***DES cipher and reverse cipher for the first approach***

6.2.3 Continued

Algorithm 6.1 *Pseudocode for DES cipher*

```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round!=16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}
```

Algorithm 6.1 *Pseudocode for DES cipher* (Continued)

```
mixer (leftBlock[48], rightBlock[48], RoundKey[48])
{
    copy (32, rightBlock, T1)
    function (T1, RoundKey, T2)
    exclusiveOr (32, leftBlock, T2, T3)
    copy (32, T3, rightBlock)
}

swapper (leftBlock[32], rigthBlock[32])
{
    copy (32, leftBlock, T)
    copy (32, rightBlock, leftBlock)
    copy (32, T, rightBlock)
}
```

6.2.3 Continued

Algorithm 6.1 *Pseudocode for DES cipher* (Continued)

```
function (inBlock[32], RoundKey[48], outBlock[32])
{
    permute (32, 48, inBlock, T1, ExpansionPermutationTable)
    exclusiveOr (48, T1, RoundKey, T2)
    substitute (T2, T3, SubstituteTables)
    permute (32, 32, T3, outBlock, StraightPermutationTable)
}
```

Algorithm 6.1 *Pseudocode for DES cipher* (Continued)

```
substitute (inBlock[32], outBlock[48], SubstitutionTables[8, 4, 16])
{
    for (i = 1 to 8)
    {
        row ← 2 × inBlock[i × 6 + 1] + inBlock [i × 6 + 6]
        col ← 8 × inBlock[i × 6 + 2] + 4 × inBlock[i × 6 + 3] +
              2 × inBlock[i × 6 + 4] + inBlock[i × 6 + 5]

        value = SubstitutionTables [i][row][col]

        outBlock[[i × 4 + 1] ← value / 8;      value ← value mod 8
        outBlock[[i × 4 + 2] ← value / 4;      value ← value mod 4
        outBlock[[i × 4 + 3] ← value / 2;      value ← value mod 2
        outBlock[[i × 4 + 4] ← value
    }
}
```

6.2.3 Continued

Alternative Approach

We can make all 16 rounds the same by including one swapper to the 16th round and add an extra swapper after that (two swappers cancel the effect of each other).

Key Generation

The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key.

6.2.3 Continued

Shifting

Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits

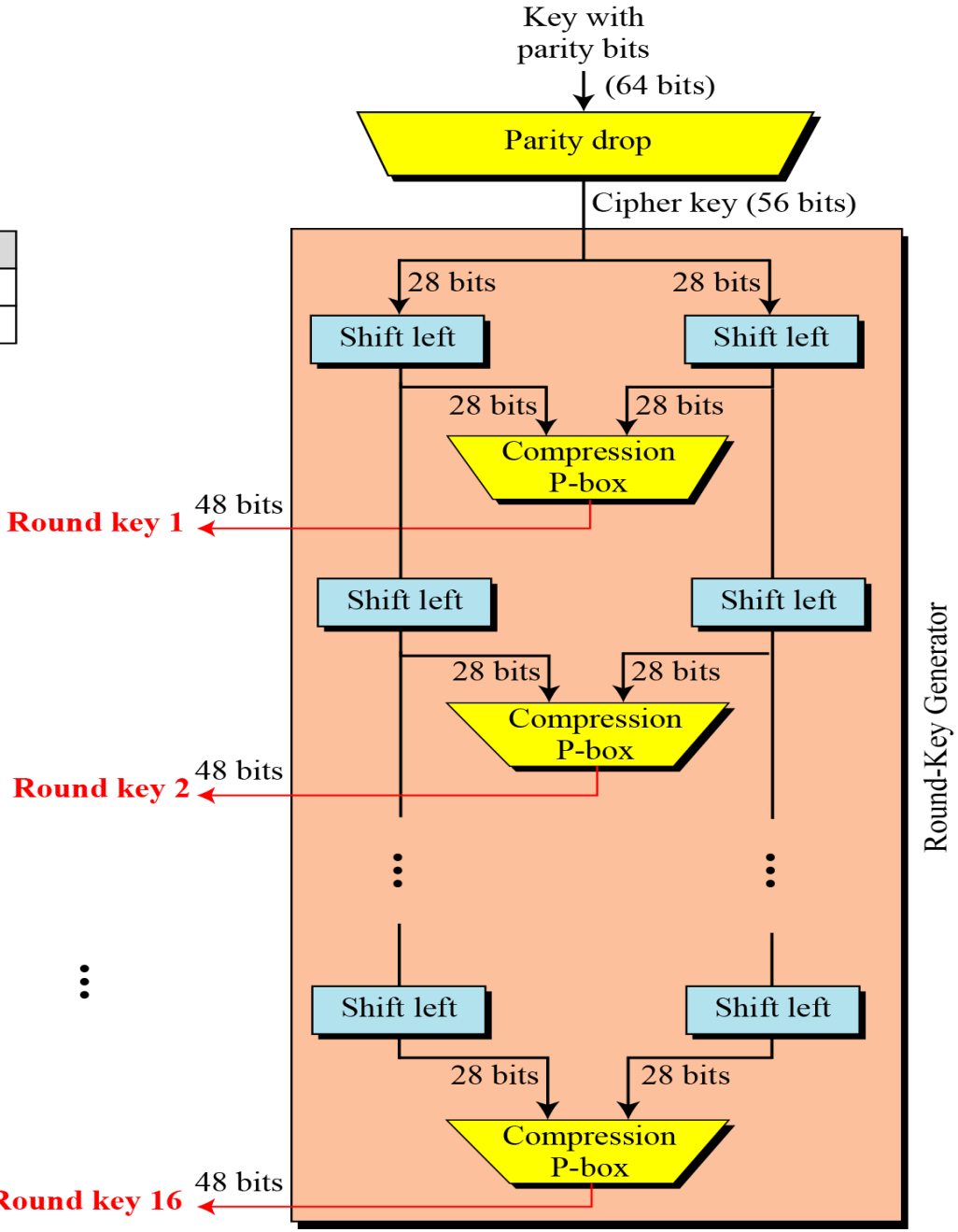


Figure 6.10

Key generation

6.2.3 Continued

Table 6.12 *Parity-bit drop table*

57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	38
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04

Table 6.13 *Number of bits shifts*

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Table 6.14 *Key-compression table*

14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

6.2.3 Continued

Algorithm 6.2 *Algorithm for round-key generation*

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
    permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
    split (56, 28, cipherKey, leftKey, rightKey)
    for (round = 1 to 16)
    {
        shiftLeft (leftKey, ShiftTable[round])
        shiftLeft (rightKey, ShiftTable[round])
        combine (28, 56, leftKey, rightKey, preRoundKey)
        permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
    }
}
```

Algorithm 6.2 *Algorithm for round-key generation (Continue)*

```
shiftLeft (block[28], numOfShifts)
{
    for (i = 1 to numOfShifts)
    {
        T ← block[1]
        for (j = 2 to 28)
        {
            block [j−1] ← block [j]
        }
        block[28] ← T
    }
}
```

6.2.4 Examples

Example 6.5

We choose a random plaintext block and a random key, and determine what the ciphertext block would be (all in hexadecimal):

Plaintext: 123456ABCD132536 Key: AABB09182736CCDD
CipherText: C0B7A8D05F3A829C

Table 6.15 *Trace of data for Example 6.5*

Plaintext: 123456ABCD132536			
After initial permutation: 14A7D67818CA18AD After splitting: L_0=14A7D678 R_0=18CA18AD			
Round	*Left*	*Right*	*Round Key*
Round 1	18CA18AD	5A78E394	194CD072DE8C
Round 2	5A78E394	4A1210F6	4568581ABCCE
Round 3	4A1210F6	B8089591	06EDA4ACF5B5
Round 4	B8089591	236779C2	DA2D032B6EE3

Example 6.5 Continued

Table 6.15 *Trace of data for Example 6.5 (Conintued*

Round 5	236779C2	A15A4B87	69A629FEC913
Round 6	A15A4B87	2E8F9C65	C1948E87475E
Round 7	2E8F9C65	A9FC20A3	708AD2DDB3C0
Round 8	A9FC20A3	308BEE97	34F822F0C66D
Round 9	308BEE97	10AF9D37	84BB4473DCCC
Round 10	10AF9D37	6CA6CB20	02765708B5BF
Round 11	6CA6CB20	FF3C485F	6D5560AF7CA5
Round 12	FF3C485F	22A5963B	C2C1E96A4BF3
Round 13	22A5963B	387CCDAA	99C31397C91F
Round 14	387CCDAA	BD2DD2AB	251B8BC717D0
Round 15	BD2DD2AB	CF26B472	3330C5D9A36D
Round 16	19BA9212	CF26B472	181C5D75C66D
After combination: 19BA9212CF26B472			
Ciphertext: C0B7A8D05F3A829C			*(after final permutation)*

6.2.4 Continued

Example 6.6

Let us see how Bob, at the destination, can decipher the ciphertext received from Alice using the same key. Table 6.16 shows some interesting points.

Ciphertext: C0B7A8D05F3A829C			
After initial permutation: 19BA9212CF26B472 After splitting: L_0=19BA9212 R_0=CF26B472			
Round	*Left*	*Right*	*Round Key*
Round 1	CF26B472	BD2DD2AB	181C5D75C66D
Round 2	BD2DD2AB	387CCDAA	3330C5D9A36D
. . .	. . .	. . .	. . .
Round 15	5A78E394	18CA18AD	4568581ABCCE
Round 16	14A7D678	18CA18AD	194CD072DE8C
After combination: 14A7D67818CA18AD			
Plaintext:123456ABCD132536 (after final permutation)			

6-3 DES ANALYSIS

Critics have used a strong magnifier to analyze DES. Tests have been done to measure the strength of some desired properties in a block cipher.

Topics discussed in this section:

6.3.1 Properties
6.3.2 Design Criteria
6.3.3 DES Weaknesses

6.3.1 Properties



Two desired properties of a block cipher are the avalanche effect and the completeness.

Example 6.7

To check the avalanche effect in DES, let us encrypt two plaintext blocks (with the same key) that differ only in one bit and observe the differences in the number of bits in each round.

Plaintext: 0000000000000000 Key: 22234512987ABB23
Ciphertext: 4789FD476E82A5F1

Plaintext: 0000000000000001 Key: 22234512987ABB23
Ciphertext: 0A4ED5C15A63FEA3

6.3.1 Continued

Example 6.7 Continued

Although the two plaintext blocks differ only in the rightmost bit, the ciphertext blocks differ in 29 bits. This means that changing approximately 1.5 percent of the plaintext creates a change of approximately 45 percent in the ciphertext.

Table 6.17 *Number of bit differences for Example 6.7*

Rounds	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*	*16*
Bit differences	1	6	20	29	30	33	32	29	32	39	33	28	30	31	30	29

6.3.1 Continued

Completeness effect

Completeness effect means that each bit of the ciphertext needs to depend on many bits on the plaintext.

Ex: result of p-box and S-box.

6.3.2 Design Criteria

S-Boxe

The design provides confusion and diffusion of bits from each round to the next.

P-Boxes

They provide diffusion of bits.

Number of Rounds

DES uses sixteen rounds of Feistel ciphers. the ciphertext is thoroughly a random function of plaintext and ciphertext.

6.3.3 DES Weaknesses

During the last few years critics have found some weaknesses in DES.

Weaknesses in Cipher Design

1. ***Weaknesses in S-boxes***
2. ***Weaknesses in P-boxes***
3. ***Weaknesses in Key***

Table 6.18 *Weak keys*

Keys before parities drop (64 bits)	*Actual key (56 bits)*
0101 0101 0101 0101	0000000 0000000
1F1F 1F1F 0E0E 0E0E	0000000 FFFFFFF
E0E0 E0E0 F1F1 F1F1	FFFFFFF 0000000
FEFE FEFE FEFE FEFE	FFFFFFF FFFFFFF

6.3.3 Continued

Example 6.8

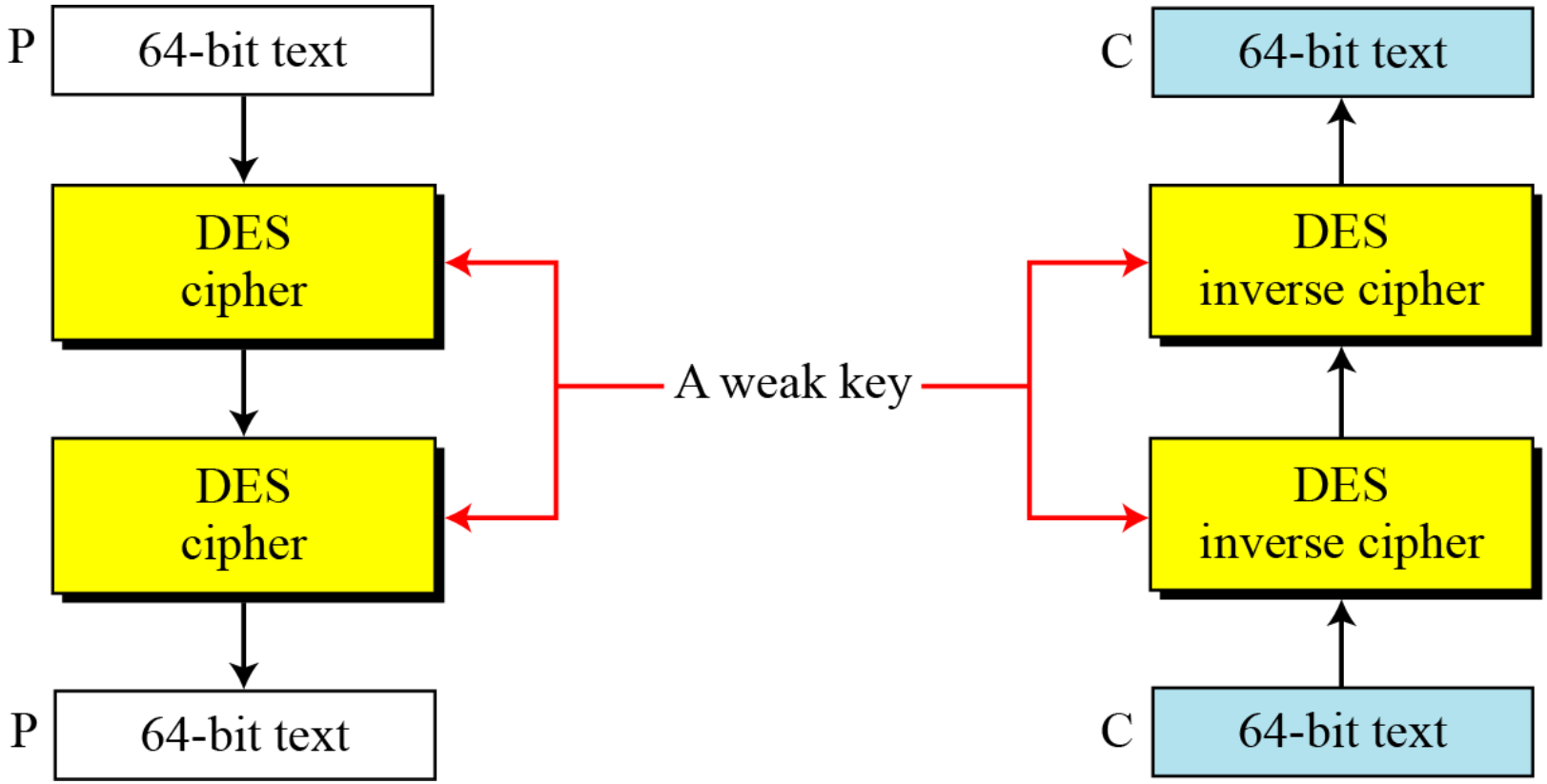
Let us try the first weak key in Table 6.18 to encrypt a block two times.
After two encryptions with the same key the original plaintext block is created.

Key: 0x0101010101010101
Plaintext: *0x123456788765432 1* Ciphertext: 0x814FE938589154F7

Key: 0x0101010101010101
Plaintext: 0x814FE938589154F7 Ciphertext: *0x123456788765432 1*

Note that we have used the encryption algorithm two times, not one encryption followed by another decryption.

Figure 6.11 *Double encryption and decryption with a weak key*



A semi-weak key creates only two different round keys

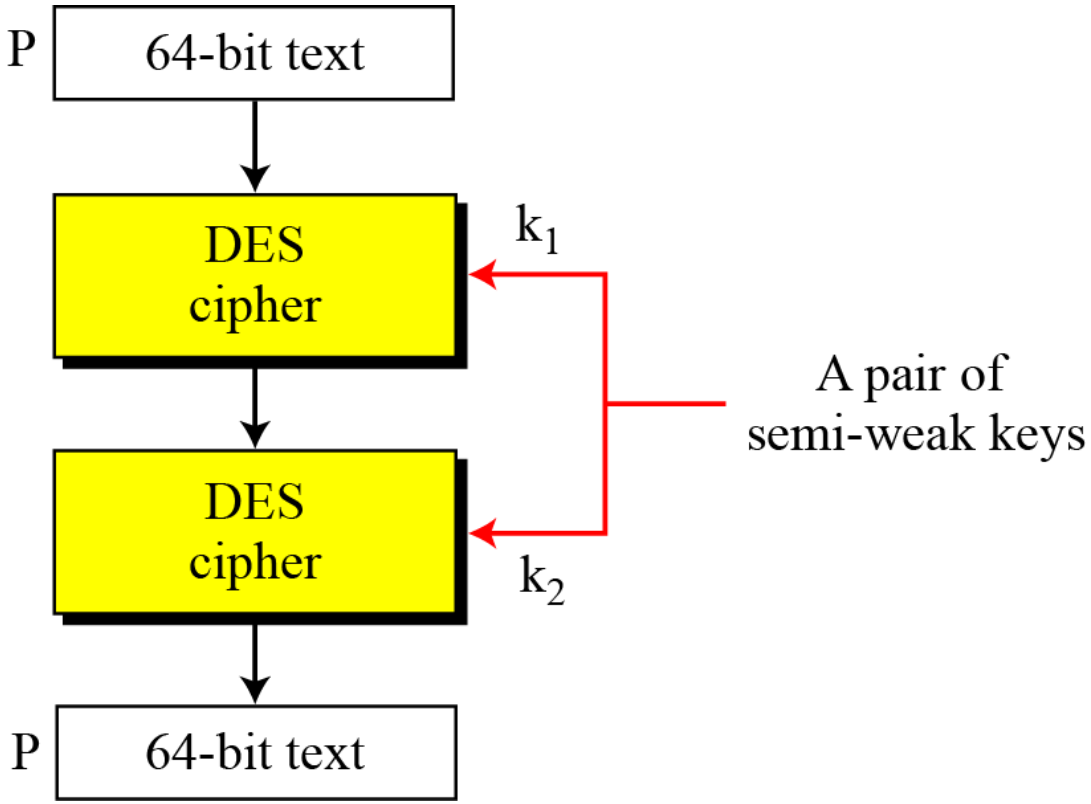
Table 6.19 *Semi-weak keys*

First key in the pair	*Second key in the pair*
01FE 01FE 01FE 01FE	FE01 FE01 FE01 FE01
1FE0 1FE0 0EF1 0EF1	E01F E01F F10E F10E
01E0 01E1 01F1 01F1	E001 E001 F101 F101
1FFE 1FFE 0EFE 0EFE	FE1F FE1F FE0E FE0E
011F 011F 010E 010E	1F01 1F01 0E01 0E01
E0FE E0FE F1FE F1FE	FEE0 FEE0 FEF1 FEF1

6.3.3 Continued

Round key 1	9153E54319BD	6EAC1ABCE642
Round key 2	6EAC1ABCE642	9153E54319BD
Round key 3	6EAC1ABCE642	9153E54319BD
Round key 4	6EAC1ABCE642	9153E54319BD
Round key 5	6EAC1ABCE642	9153E54319BD
Round key 6	6EAC1ABCE642	9153E54319BD
Round key 7	6EAC1ABCE642	9153E54319BD
Round key 8	6EAC1ABCE642	9153E54319BD
Round key 9	9153E54319BD	6EAC1ABCE642
Round key 10	9153E54319BD	6EAC1ABCE642
Round key 11	9153E54319BD	6EAC1ABCE642
Round key 12	9153E54319BD	6EAC1ABCE642
Round key 13	9153E54319BD	6EAC1ABCE642
Round key 14	9153E54319BD	6EAC1ABCE642
Round key 15	9153E54319BD	6EAC1ABCE642
Round key 16	6EAC1ABCE642	9153E54319BD

Figure 6.12 *A pair of semi-weak keys in encryption and decryption*



Example 6.9

What is the probability of randomly selecting a weak, a semi-weak, or a possible weak key?

Solution

DES has a key domain of 2^{56}. The total number of the above keys are 64 (4 + 12 + 48). The probability of choosing one of these keys is 8.8×10^{-16}, almost impossible.

6-4 Multiple DES

The major criticism of DES regards its key length. Fortunately DES is not a group. This means that we can use double or triple DES to increase the key size.

Topics discussed in this section:

6.4.1 Double DES
6.4.4 Triple DES

6.4.1 Double DES

The first approach is to use double DES (2DES).

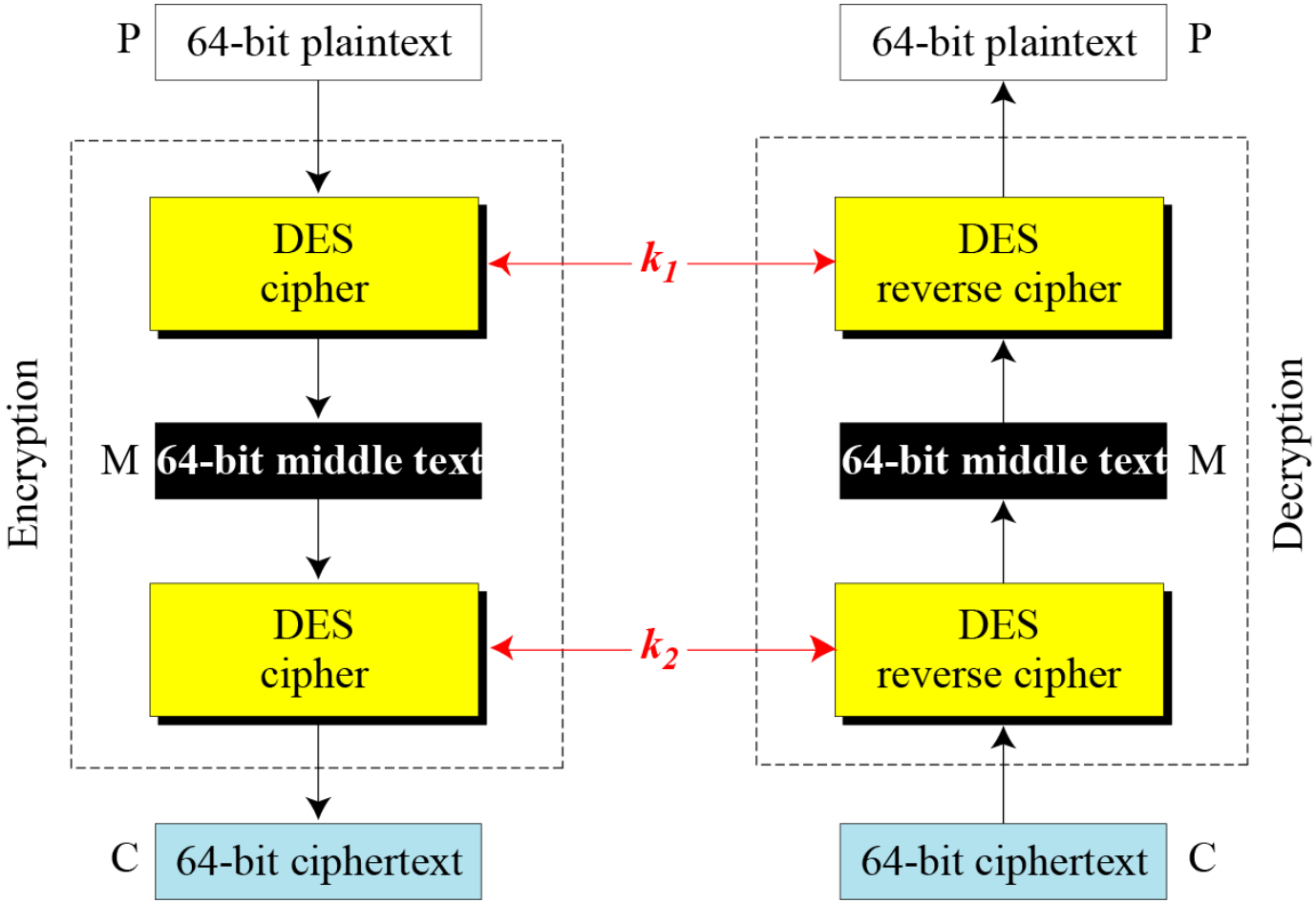
Ciphertext $C = E_{K2}(E_{K1}(P))$

Plaintext $P = D_{K1}(D_{K2}(C))$

Meet-in-the-Middle Attack

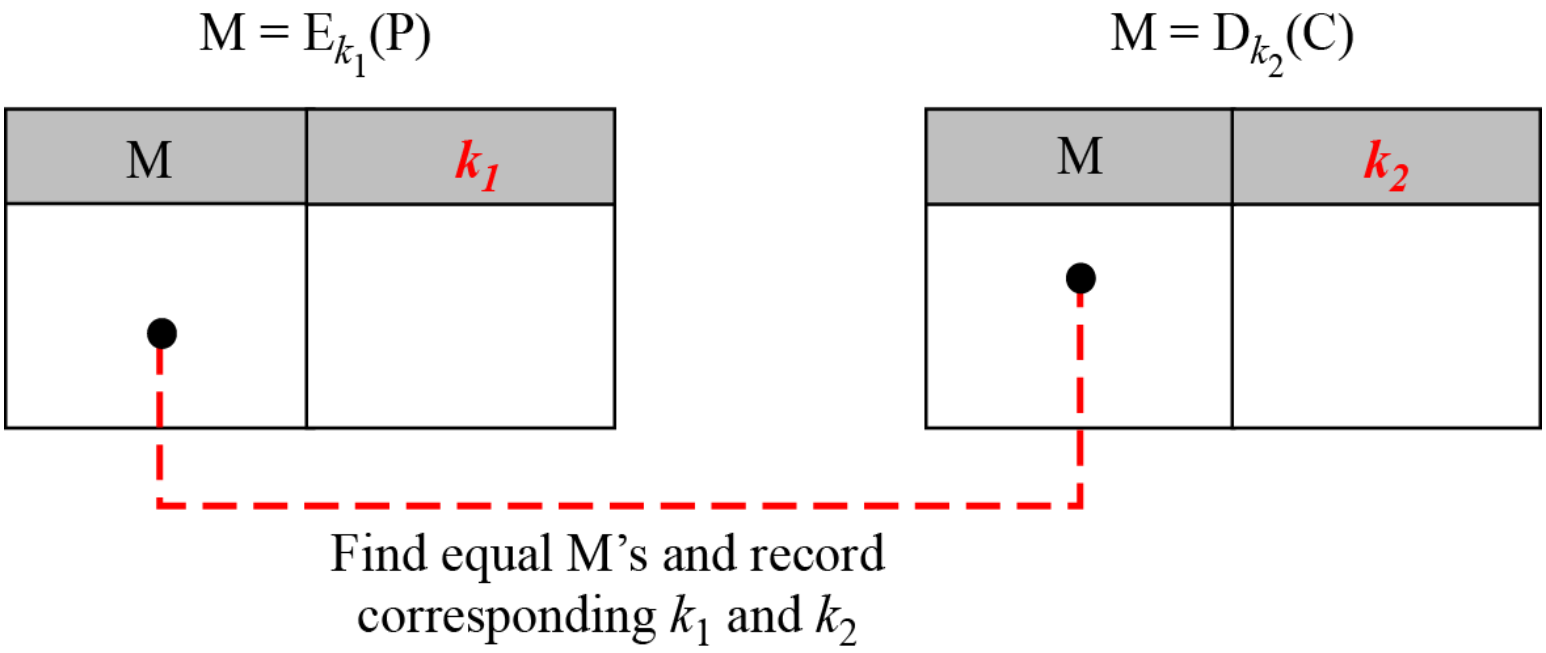
However, using a known-plaintext attack called meet-in-the-middle attack proves that double DES improves this vulnerability slightly (to 2^{57} tests), but not tremendously (to 2^{112}).

Figure 6.14 *Meet-in-the-middle attack for double DES*



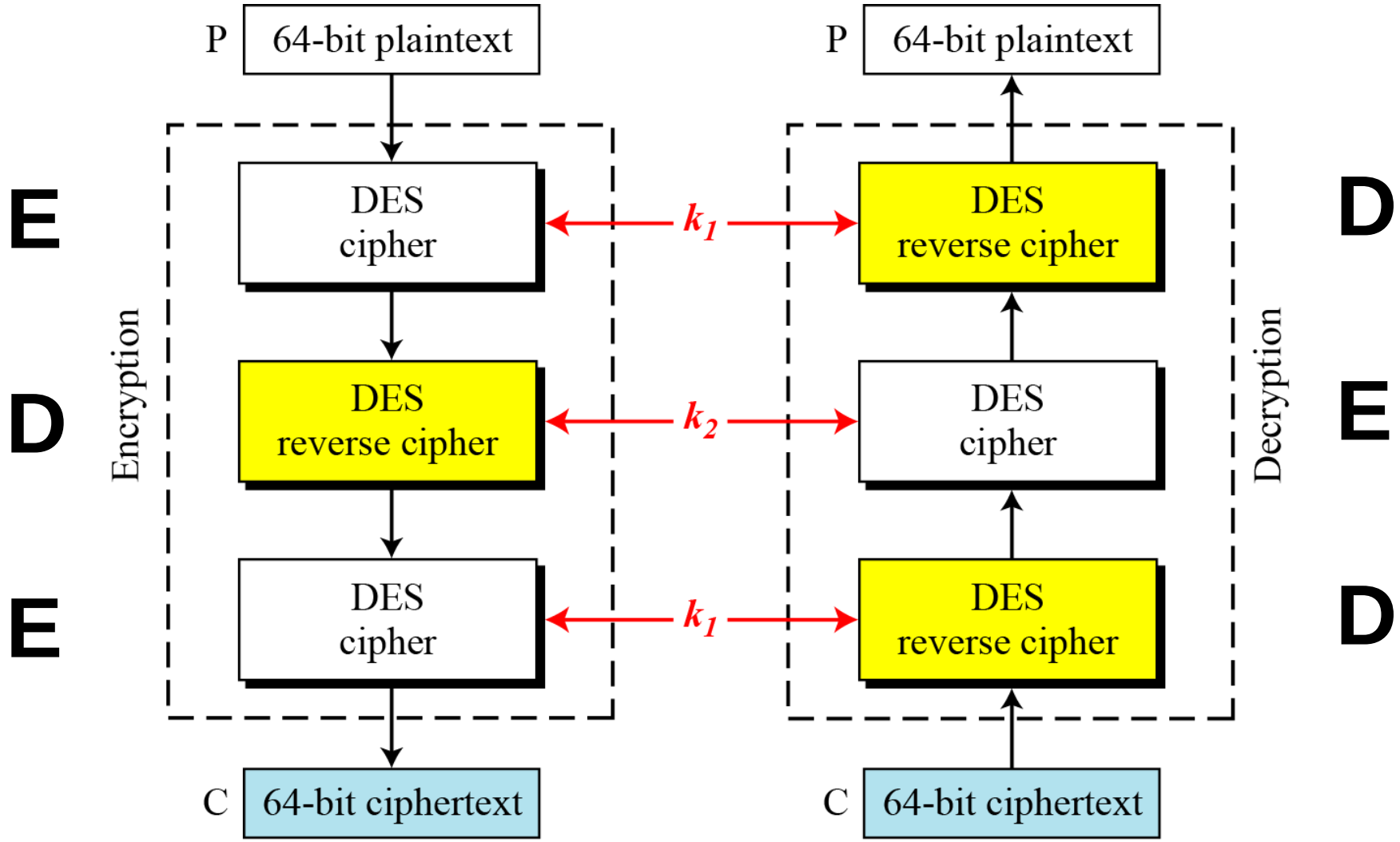
6.4.1 Continued

Figure 6.15 *Tables for meet-in-the-middle attack*



6.4.2 Triple DES

Figure 6.16 *Triple DES with two keys*



6.4.2 Continuous

Triple DES with Three Keys

The possibility of known-plaintext attacks on triple DES with two keys has enticed some applications to use triple DES with three keys. Triple DES with three keys is used by many applications such as PGP (See Chapter 16, forouzan).

6-5 Security of DES

DES, as the first important block cipher, has gone through much scrutiny. Among the attempted attacks, three are of interest: brute-force, differential cryptanalysis, and linear cryptanalysis.

Topics discussed in this section:

6.5.1 Brute-Force Attack
6.5.2 Differential Cryptanalysis
6.5.3 Linear Cryptanalysis

6.5.1 Brute-Force Attack

We have discussed the weakness of short cipher key in DES. Combining this weakness with the key complement weakness, it is clear that DES can be broken using 2^{55} encryptions.

6.5.2 Differential Cryptanalysis

It has been revealed that the designers of DES already knew about this type of attack and designed S-boxes and chose 16 as the number of rounds to make DES specifically resistant to this type of attack.

Note

We show an example of DES differential cryptanalysis in Appendix N.

6.5.3 Linear Cryptanalysis

Linear cryptanalysis is newer than differential cryptanalysis. DES is more vulnerable to linear cryptanalysis than to differential cryptanalysis. S-boxes are not very resistant to linear cryptanalysis. It has been shown that DES can be broken using 2^{43} pairs of known plaintexts. However, from the practical point of view, finding so many pairs is very unlikely.

Note

We show an example of DES linear cryptanalysis in Appendix N.